1 Project Information

Project Title: Speech Recording Denoising with Neural Networks
Team Members: Skylar Denno (only)
GitHub Link: https://github.com/skylar-slideshows/ML_denoise

2 Dataset Summary

Dataset Name: Noisy speech database for training speech enhancement algorithms and TTS models (28 speaker trainset and noisy testset)
Dataset Source: University of Edinburgh: DataShare, School of Informatics, Centre for Speech Technology Research (CSTR)
Dataset Link: https://doi.org/10.7488/ds/2117
Number of Files: 11,572 (pairs of audio files: one clean, one with noise)
Number of Samples: The total number of short-time Fourier transform (STFT) frames across all the audio files. Each frame is 512 audio samples long (32ms each at the training sample rate of 16kHz).
Target: Parameters (values on each of the 257 frequency bins for the spectral mask to filter out) per frame of audio.

3 Data Preprocessing

For this project, we want our model to "predict" the best mask values to use to mask noise, across the 257 frequency ranges that cover the audio spectrum. This model will not be "generating" clean audio from dirty audio. It just predicts the best filter parameters to use, and using DSP, that filter/mask is applied.

The dataset we are using is already well cleaned, but a set of validation functions were written to flag and drop any files with issues, which is especially useful if someone wants to train this model on a different dataset in the future.

This dataset is organized into two folders: a "clean" folder containing 11,572 audio files with clean/noiseless speech recordings, and a "noisy" folder containing the same audio recordings, with noise added to each. Each file in the clean folder should have a noisy partner in the noisy folder. Any files without matching clean and noisy versions are essentially

My preprocessing code first flags any files that do not have a partner in one of the folders. To test, I temporarily removed 8 of the audio files from the clean folder, but not the noisy folder. Upon running the preprocessing script, this was the output:

```
(base) skylar@Skybook-Pro ML_denoise % python src/1_preprocess.py
WARNING: 8 unpaired files (no partner) will be skipped.

===== SUMMARY =====
Paired inputs      : 11564
Pairs KEPT         : 11564
Pairs DROPPED      : 0
Unpaired skipped   : 8

Copied to:
  data/preprocess/clean_out
  data/preprocess/noisy_out
Report: data/preprocess/report.csv
(base) skylar@Skybook-Pro ML_denoise %
```

Figure 1: Preprocessing script output after severing 8 pairs

All the files, except ones without partners, were copied to a new preprocessed data folder.

Next, any silent audio files (containing no samples louder than -40dBFS (decibels full scale) should be removed. To test this, I added two silent audio files each, with the same names, to the clean and noisy folders and ran the script:

```
(base) skylar@Skybook-Pro ML_denoise % python src/1_preprocess.py

===== SUMMARY =====
Paired inputs      : 11574
Pairs KEPT         : 11572
Pairs DROPPED      : 2
Unpaired skipped   : 0
Drop reasons (pairs affected):
    quiet       : 2

Copied to:
  data/preprocess/clean_out
  data/preprocess/noisy_out
Report: data/preprocess/report.csv
(base) skylar@Skybook-Pro ML_denoise %
```

Figure 2: Preprocessing script output after adding 2 pairs containing silence

	set	file	kept	flags	sr	channels	duration_s	rms_dbfs	peak_dbfs	dc_offset	clip_frac
2	clean	SILENCE copy.wav	FALSE	quiet	48000	1	30	-84.13	-71.22	0	0
3	noisy	SILENCE copy.wav	FALSE	quiet	48000	1	30	-84.13	-71.22	0	0
4	clean	SILENCE.wav	FALSE	quiet	48000	1	30	-84.13	-71.22	0	0
5	noisy	SILENCE.wav	FALSE	quiet	48000	1	30	-84.13	-71.22	0	0
6	clean	p226_001.wav	TRUE		48000	1	2.28	-23.74	-6.02	-4.00E-05	0
7	noisy	p226_001.wav	TRUE		48000	1	2.28	-23.47	-5.96	-4.00E-05	0

Figure 3: Preprocessing script report.csv after adding 2 pairs containing silence

The report.csv output showed the 2 pairs that were not copied to the preprocessed data folder. The reason is given as "quiet."

Next, for very loud signals (where a large proportion of audio samples are near the maximum amplitudes at -1 and 1), we want to remove them, as this digital clipping, or shaving off of the peaks of the waves, causes distortion and will be bad to train the model on. I placed two identical files of full-scale square wave at 440Hz (containing samples only at -1 and 1) in the clean and noisy folders. I ran the preprocessing script:

```
(base) skylar@Skybook-Pro ML_denoise % python src/1_preprocess.py

===== SUMMARY =====
Paired inputs      : 11574
Pairs KEPT         : 11572
Pairs DROPPED      : 2
Unpaired skipped   : 0
Drop reasons (pairs affected):
    clipping    : 2

Copied to:
  data/preprocess/clean_out
  data/preprocess/noisy_out
Report: data/preprocess/report.csv
(base) skylar@Skybook-Pro ML_denoise %
```

Figure 4: Preprocessing script output after adding 2 pairs containing clipping

set	file	kept	flags	sr	channels	duration_s	rms_dbfs	peak_dbfs	dc_offset	clip_frac
clean	CLIP copy.wav	FALSE	clipping	48000	1	30	-0.17	0	0.00052	0.512749
noisy	CLIP copy.wav	FALSE	clipping	48000	1	30	-0.17	0	0.00052	0.512749
clean	CLIP.wav	FALSE	clipping	48000	1	30	-0.17	0	0.00052	0.512749
noisy	CLIP.wav	FALSE	clipping	48000	1	30	-0.17	0	0.00052	0.512749
clean	p226_001.wav	TRUE		48000	1	2.28	-23.74	-6.02	-4.00E-05	0
noisy	p226_001.wav	TRUE		48000	1	2.28	-23.47	-5.96	-4.00E-05	0

Figure 5: Preprocessing script report.csv after adding 2 pairs containing clipping

The script similarly detected and flagged these files, keeping them out of the preprocessed data folder.

After this, I created two audio files containing white noise at a medium volume (to not trigger clipping or silence flags), but with durations of 163ms (below the 1s minimum length threshold). I also accidentally exported these files with a 44.1kHz sample rate, which also caused the bad_sr flag.

```
(base) skylar@Skybook-Pro ML_denoise % python src/1_preprocess.py

===== SUMMARY =====
Paired inputs      : 11574
Pairs KEPT         : 11572
Pairs DROPPED      : 2
Unpaired skipped   : 0
Drop reasons (pairs affected):
    short       : 2
    bad_sr      : 2

Copied to:
  data/preprocess/clean_out
  data/preprocess/noisy_out
Report: data/preprocess/report.csv
(base) skylar@Skybook-Pro ML_denoise %
```

Figure 6: Preprocessing script report.csv after adding 2 pairs of very short audio (44.1kHz)

set	file	kept	flags	sr	channels	duration_s	rms_dbfs	peak_dbfs	dc_offset	clip_frac
clean	SHORT copy.wav	FALSE	bad_sr | short	44100	1	0.163	-6.75	-1.94	0.0049	0
noisy	SHORT copy.wav	FALSE	bad_sr | short	44100	1	0.163	-6.75	-1.94	0.0049	0
clean	SHORT.wav	FALSE	bad_sr | short	44100	1	0.163	-6.75	-1.94	0.0049	0
noisy	SHORT.wav	FALSE	bad_sr | short	44100	1	0.163	-6.75	-1.94	0.0049	0
clean	p226_001.wav	TRUE		48000	1	2.28	-23.74	-6.02	-4.00E-05	0
noisy	p226_001.wav	TRUE		48000	1	2.28	-23.47	-5.96	-4.00E-05	0

Figure 7: Preprocessing script report.csv after adding 2 pairs of very short audio (44.1kHz)

The script similarly detected and flagged these files, keeping them out of the preprocessed data folder.

Tests of these are not shown here, but my preprocessing script also checks for files with the wrong sample rate (48kHz is expected), and it checks for files with a DC offset (the mean value of all the samples is significantly different than 0; the audio is not symmetrical about 0).

I then ran this preprocessing script on the real dataset, without any of my added testing files. The script indicated that all pairs were kept, confirming that no files contained audio data that would be an issue for training on.

```
(base) skylar@Skybook-Pro ML_denoise % python src/1_preprocess.py

===== SUMMARY =====
Paired inputs      : 11572
Pairs KEPT         : 11572
Pairs DROPPED      : 0
Unpaired skipped   : 0

Copied to:
  data/preprocess/clean_out
  data/preprocess/noisy_out
Report: data/preprocess/report.csv
```

Figure 8: Preprocessing script output for the actual dataset.

To make my end product as easy to use and reproduce as possible, I included individual config files for each of the scripts in the project folder. In the preprocessing config, the thresholds to filter good and bad files can be easily changed, as well as the file paths for the data input and output folders.

```yaml
# CONFIG FILE FOR 1_preprocess.py

# File paths to read and save
clean_in: data/raw/clean_trainset_28spk_wav
noisy_in: data/raw/noisy_trainset_28spk_wav
clean_out: data/preprocess/clean_out
noisy_out: data/preprocess/noisy_out      You, 5 hours ago • save progress …
report: data/preprocess/report.csv

# File thresholds
min_duration: 1.0 # minimum file duration in seconds
min_dbfs: -40.0 # minimum signal level in decibels full scale
clip_dbfs: -0.1 # maximum signal level before flagging as clipping
clip_frac: 0.001 # maximum fraction of samples allowed to exceed CLIP_DBFS
max_dc_offset: 0.05 # maximum allowed DC offset in the signal
expected_channels: 1 # expected number of audio channels (mono)
expected_sr: 48000 # expected sample rate of the audio files 48kHz

# which flags cause a file (and therefore its whole pair) to be dropped.
drop_flags:
  - short
  - quiet
  - clipping
  - offset
  - bad_sr
```

Figure 9: 1_preprocess.yaml Config file

Finally, the ML model will not be taking in the array of sample values directly from the audio file, but instead, a set of complex values of the Fourier transform of each audio file. This is because the noise masking problem is inherently frequency based, so a Fourier transformed set of data makes much more sense for this application. However, the problem is also time based, since the noise changes over time, and a single fixed mask would not be sufficient, so we are not using a regular Fourier transform. Instead we are using a short time Fourier transform (STFT), which breaks the signal up into small frames, before applying the transform to each frame separately.

Therefore, a script "2_encode.py" was created to take in the preprocessed data files (still separated into clean and noisy folders) and create corresponding STFT arrays for each file. The arrays would contain as many columns are there are frequency bins for each frame of audio, and as many rows as there are frames for that specific file. Please note that this script is compute-intensive and may take a while to run locally (about 10 minutes on a 2026 M5 Macbook Pro).

```
(base) skylar@Skybook-Pro ML_denoise % python -c "import numpy as np; d=np.load('data/encode/cache/p226_001.npz'); print
(d.files); [print(k, d[k].shape, d[k].dtype) for k in d.files]"

['clean_stft', 'noisy_stft', 'gain']
clean_stft (257, 286) complex64
noisy_stft (257, 286) complex64
gain () float32
```

Figure 10: Checking the array shape of one of the encoded data files. The first dimension (257) is the number of frequency bins the audio is divided into by the STFT (257 complex numbers = 514 individual coefficients - real part and imaginary of each). For this particular file, there are 286 time frames (each 32ms, meaning this file was about 9.15 seconds long)

Finally, after encoding, a make_split script was written to assign some of the speakers to a validation/test set, and the rest to the training set. I made the decision to split at the speaker level, rather than the individual file level so the model would not see any of the test speakers' files during training. This prevents the model from learning from the specific qualities of the speakers' voices, which would make the testing look better than it would actually perform in genuine trials, where none of the model's training set speakers match the real-life speaker's exact vocal characteristics.

```
===== SPLIT =====
Val speakers (4): ['p226', 'p230', 'p274', 'p278']
Train: 9964 utterances from 24 speakers
Val  : 1608 utterances from 4 speakers
Speaker overlap (must be empty): set()

Wrote:
  data/encode/cache/train_files.txt
  data/encode/cache/val_files.txt
(base) skylar@Skybook-Pro ML_denoise %
```

Figure 11: 3_make_split.py script output. This script uses a seed (set in its config file) to ensure reproducability. For now, we are defaulting to a ratio of 4 validation speakers, 24 training speakers

The file names for each set are written to separate text documents, and the ML training script just needs to use the corresponding files, located in the encoding folder, containing the STFT arrays.

The data is now ready to be used in the training process.

4 Choice of Machine Learning Model

Since this kind of noise processing is heavily context and sequence dependent (what counts as "noise" in the audio depends on what is considered "noise" on nearby/previous frames of audio), it was natural to use a recursive neural network. Recursive Neural Networks (RNNs) are often used for audio problems like this, where the data is temporal and sequential, therefore requiring a model with "memory" of past inputs.

Next, we want to pick a specific RNN variant, since RNNs cover a vast range of different models. For this project, a long-short term memory network is chosen (LSTM). This is because of the nature of the data we are using. Each individual speech clip may be several seconds long, but each unit for time recursion: individual audio frames of 32ms / 512 samples are very short. There will be hundreds per file, so when the model needs to figure out the relationship between the current frame and the past hundreds of frames, the problem of vanishing gradients will become significant. If hundreds of gradient values (usually less than 1) are multiplied together during back propagation, the resulting gradient will be nearly 0, and the model will not be able to "understand" the relationship between the current frame and much earlier frames of audio.

Since LSTMs use cell states which additively combines previous gradients, rather than multiplicatively, they are the best choice for a problem like this, where we will often need context from hundreds of frames and gradients in the past. I plan to use the Python PyTorch "torch.nn.lstm" class to implement this model.

This model will be used under the "many-to-many" framework, as each trial will take in numerous frames of audio from an audio file and output numerous frames of mask values (257 per frame) to apply using DSP, in order to mask the audio.

For more specifics, I will probably use the standard two layers of recursion (may be changed when actually doing training/testing), a starting dropout of 0 (subject to change if overfitting occurs), and unidirectionality, since only caring about past frames, and not what happens in the future is simpler to start with, and can be used in live/real-time processing systems, such as conference call software.

4.1 Evaluation Metrics

There is still the question of how we will evaluate the model's performance quantitatively, since just "listening" to the test output and saying it sounds less noisy is not enough. Therefore we will evaluate denoising performance using three standard objective metrics, each capturing a different aspect of speech quality.

4.1.1 PESQ (Perceptual Evaluation of Speech Quality)

PESQ estimates the perceptual quality of the enhanced speech, i.e. how natural and pleasant it sounds to a human listener. It compares the enhanced signal against the clean reference using a model of human auditory perception, accounting for how the ear responds to different frequencies and loudness levels. PESQ scores typically range from -0.5 to 4.5, where higher values indicate better perceived quality. Because it models perception rather than raw signal error, PESQ correlates reasonably well with human opinion scores.

4.1.2 STOI (Short-Time Objective Intelligibility)

STOI measures intelligibility (how understandable the speech is) which is distinct from how pleasant it sounds. It compares the short-time temporal envelopes of the enhanced and clean signals across frequency bands, since intelligibility depends largely on preserving these envelope patterns. STOI produces a score between 0 and 1 (often reported as a percentage), where higher values indicate more intelligible speech. A signal can sound distorted yet remain highly intelligible, so STOI captures information that PESQ does not. We don't want each filter mask to be very different from frame to frame, producing a garbled, highly varying output that is hard to understand.

4.1.3 SI-SDR (Scale-Invariant Signal-to-Distortion Ratio)

SI-SDR measures raw signal fidelity (how close the enhanced waveform is to the clean reference) expressed in decibels. It decomposes the estimate into a component aligned with the target and a residual distortion component, and reports their ratio. The scale-invariant property means the metric is unaffected by overall amplitude scaling, so it rewards correct signal shape rather than matching loudness. Higher values (in dB) indicate less distortion. Unlike PESQ and STOI, SI-SDR is a purely mathematical measure and does not model human perception.

4.1.4 Why Use All Three

The three metrics capture complementary aspects: PESQ reflects perceptual quality, STOI reflects intelligibility, and SI-SDR reflects signal fidelity. A model may improve one while degrading another. For example, aggressive noise suppression can raise SI-SDR while introducing artifacts that lower PESQ. Reporting all three provides a more complete picture of denoising performance than any single metric alone.

5 Updated Research Questions

1. How should denoising performance be evaluated, given that objective metrics (PESQ, STOI, SI-SDR) are proxies for the true goal of human-perceived speech quality and intelligibility? What does each metric capture, and where might they disagree with subjective listening?

2. How does model performance vary across input signal-to-noise ratios, and does the denoiser provide greater improvement on noisier inputs than on cleaner ones?

3. Do the STFT parameters (window length and hop size) affect denoising quality, and what trade-off exists between time and frequency resolution for speech enhancement?

4. How well does the model generalize to new/real-world speakers, given that the training and test sets are speaker-disjoint?

5. Does the LSTM bottleneck improve denoising performance compared to a purely convolutional model without temporal modeling, indicating that long-range temporal context contributes to speech enhancement?

6. How does the number of LSTM layers and hidden units affect denoising quality, and is there a point of diminishing returns where added recurrent capacity no longer improves performance?

7. Does a bidirectional LSTM (which uses both past and future frames) outperform a unidirectional LSTM (past frames only), and is the quality gain worth the loss of real-time capability?